Circuit Breaker

Circuit Breaker · Protect calls to flaky rates API · Build with Waqar

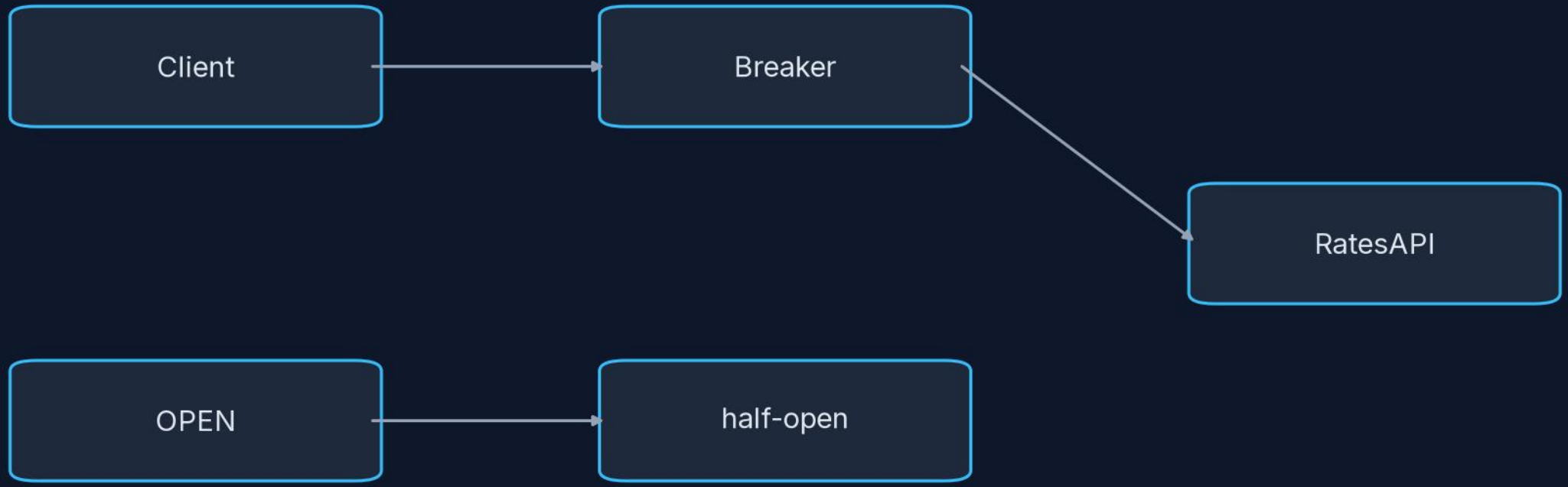
Today's Hook

- Problem: Protect calls to flaky rates API
- Pattern: Circuit Breaker — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Protect calls to flaky rates API
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Circuit Breaker: Protect calls to flaky rates API

Real-World Scenario

- Running example: Protect calls to flaky rates API
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker.problem;

import java.util.*;

public final class NaiveRates {
  private int fails;
  public String rate(){ if(fails++ < 3) throw new RuntimeException("down"); return "83.2"; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

enum CbState { CLOSED, OPEN, HALF_OPEN }
final class CircuitBreaker {
  private final int threshold; private int failures; private CbState state=CbState.CLOSED; private long openedAt;
  private final long resetMs; CircuitBreaker(int threshold, long resetMs){ this.threshold=threshold; this.resetMs=resetMs; }
  CbState state(){ return state; }
  String call(Supplier<String> action){
    if (state==CbState.OPEN){
      if (System.currentTimeMillis()-openedAt < resetMs) throw new IllegalStateException("circuit open");
      state=CbState.HALF_OPEN;
    }
    try {
      String r=action.get(); failures=0; state=CbState.CLOSED; return r;
    } catch (RuntimeException e){
      failures++;
      if (failures>=threshold || state==CbState.HALF_OPEN){ state=CbState.OPEN; openedAt=System.currentTimeMillis(); }
      throw e;
    }
```

Solution (2)

```
    }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.circuitbreaker.problem;

import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Supplier;

public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Circuit Breaker: flaky rates API ===");
    System.out.println("--- naive ---");
    var n=new NaiveRates();
    for(int i=0;i<4;i++){ try{ System.out.println(n.rate()); } catch(Exception e){ System.out.println("fail"); } }
    System.out.println("--- circuit breaker ---");
    AtomicInteger blows=new AtomicInteger();
    var cb=new CircuitBreaker(2, 10);
    Supplier<String> flaky=() -> { if(blows.getAndIncrement()<2) throw new RuntimeException("down"); return "rate=83.2"; };
    for (int i=0;i<2;i++){ try{ System.out.println(cb.call(flaky)); } catch(Exception e){ System.out.println("fail state="+cb.state()); } }
    try { cb.call(flaky); } catch(IllegalStateException e){ System.out.println("shortCircuit="+e.getMessage()+" state="+cb.state()); }
    Thread.sleep(15); blows.set(100);
    System.out.println("recovered="+cb.call(() -> "rate=83.2")+" state="+cb.state());
  }
}
```

Walkthrough Output

```
=== Circuit Breaker: flaky rates API ===
--- naive ---
fail
fail
fail
83.2
--- circuit breaker ---
fail state=CLOSED
fail state=OPEN
shortCircuit=circuit open state=OPEN
recovered=rate=83.2 state=CLOSED
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Circuit Breaker.

Related Patterns

- Contrast Circuit Breaker with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Circuit Breaker → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.